

UNIVERSIDAD ESTATAL AMAZÓNICA
FACULTAD DE RECURSOS NATURALES
CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN

DOCUMENTO Y GUÍA DE DEFENSA PRÁCTICA
PROYECTO INTEGRADOR U4 (AVANCE FINAL 16/16)

**Sistema Web de Gestión Comercial con Autenticación,
Operaciones CRUD y Base de Datos Relacional PostgreSQL**

Caso de Estudio: Ferretería “El Tornillo Dorado”

Asignatura: Desarrollo de Aplicaciones Web (2626-UEA-L-UFPTI-004-D)

Docente: Ing. Walter Nuñez, Msc.

Estudiante: Clara Anahí González Apolo

Período: 2626 – Semestre Académico

Fecha de Entrega: Octubre de 2026

Repositorio Oficial: <https://github.com/anis1605/2626---DESARROLLO-DE-APLICACIONES-WEB---UEA>

GitHub Pages: <https://anis1605.github.io/2626---DESARROLLO-DE-APLICACIONES-WEB---UEA>

Índice

1. Ficha Técnica y Objetivos del Proyecto	2
1.1. Especificaciones Técnicas	2
1.2. Estructura del Repositorio	2
2. Modelo Relacional de Base de Datos (PostgreSQL)	2
2.1. Consulta Multi-Tabla con JOIN Implementada	3
3. Módulo de Autenticación y Control de Acceso (Semana 14)	3
4. Implementación de Operaciones CRUD en la Interfaz	4
4.1. Resumen de Endpoints CRUD	9
5. Control de Versiones y Despliegue en la Nube	9
5.1. Repositorio Oficial de GitHub	9
6. Guion Técnico para la Exposición y Defensa (3 a 5 Minutos)	10

1. Ficha Técnica y Objetivos del Proyecto

El presente proyecto integrador representa la consolidación práctica de la asignatura **Desarrollo de Aplicaciones Web**. El sistema implementa una arquitectura completa basada en el microframework Flask, integrando persistencia relacional con PostgreSQL, autenticación robusta mediante Flask-Login y cifrado criptográfico Werkzeug, operaciones completas de creación, lectura, modificación y eliminación (CRUD), y un frontend moderno con Bootstrap 5 bajo una paleta pastel y tipografía Plus Jakarta Sans.

1.1. Especificaciones Técnicas

- **Lenguaje de Programación:** Python 3.12.
- **Framework Web Backend:** Flask 3.1.3 con arquitectura modular MVC.
- **Gestión de Autenticación:** Flask-Login 0.6.3 (UserMixin, LoginManager, login_required).
- **Seguridad y Hashing de Contraseñas:** Werkzeug 3.1.8 (Algoritmo script).
- **Formularios y Protección CSRF:** Flask-WTF 1.3.0 y WTForms 3.2.2.
- **Motor de Base de Datos:** PostgreSQL 15+ con conector psycopg2-binary 2.9.9.
- **Servidor WSGI de Producción:** Gunicorn 21.2.0 (configurado en Procfile).
- **Control de Versiones y Despliegue:** Git, GitHub (anis1605) y Render.

1.2. Estructura del Repositorio

A diferencia de versiones preliminares, el proyecto final se encuentra desplegado directamente en la **raíz del repositorio** para garantizar su ejecución inmediata y compatibilidad directa con Render:

```
Proyecto_Final/  
|-- app.py                # Controlador Flask, endpoints y rutas protegidas  
|-- models.py            # Modelo Usuario con UserMixin y metodos SQL  
|-- requirements.txt      # Dependencias completas del entorno  
|-- Procfile             # Configuración de despliegue: web: gunicorn app:app  
|-- conexion/conexion.py  # Conector psycopg2 con soporte local y nube (Render)  
                           )  
|-- sql/esquema.sql       # DDL de PostgreSQL: 5 tablas relacionadas y datos  
                           demo  
|-- forms/               # Formularios WTForms (Login, Usuario, Producto,  
                           Cliente...)  
|-- templates/           # Vistas Jinja2 (base.html, dashboard, login,  
                           modulos CRUD)  
'-- static/              # Hojas de estilo pastel, JavaScript y recursos  
                           graficos
```

2. Modelo Relacional de Base de Datos (PostgreSQL)

La base de datos relacional **ferreteria_db** fue diseñada para soportar un flujo comercial integral y cumple con el requisito obligatorio de incorporar **al menos 3 tablas relacionadas** mediante claves primarias (PRIMARY KEY) y foráneas (FOREIGN KEY), alcanzando un total de 5 tablas:

1. **usuarios:** Almacena las cuentas de acceso con su identificador primario (`id SERIAL`), usuario único (`usuario VARCHAR(50) UNIQUE`), contraseña cifrada en hash (`password VARCHAR(255)`) y rol.
2. **proveedores:** Entidad fuerte identificada por su RUC (`ruc VARCHAR(13) PRIMARY KEY`), razón social, categoría comercial, contacto y teléfono.
3. **productos:** Registra los ítems del catálogo. Incorpora la clave foránea `proveedor_ruc` que referencia a `proveedores(ruc)` mediante integridad referencial (`ON DELETE SET NULL ON UPDATE CASCADE`).
4. **clientes:** Directorio de compradores registrados por su RUC o Cédula (`ruc VARCHAR(13) PRIMARY KEY`), tipo, teléfono, correo y dirección.
5. **facturas:** Documentos de venta con clave foránea `cliente_ruc` asociada a `clientes(ruc)`, registrando fecha, subtotal, IVA (15 %) y total.

2.1. Consulta Multi-Tabla con JOIN Implementada

Para evidenciar el aprovechamiento de las relaciones foráneas, en el listado de productos se ejecuta la siguiente consulta parametrizada con `LEFT JOIN`:

```
SELECT p.id, p.nombre, p.categoria, p.unidad, p.stock, p.precio,
       p.proveedor_ruc, pr.empresa AS proveedor_empresa
FROM productos p
LEFT JOIN proveedores pr ON p.proveedor_ruc = pr.ruc
ORDER BY p.nombre ASC;
```

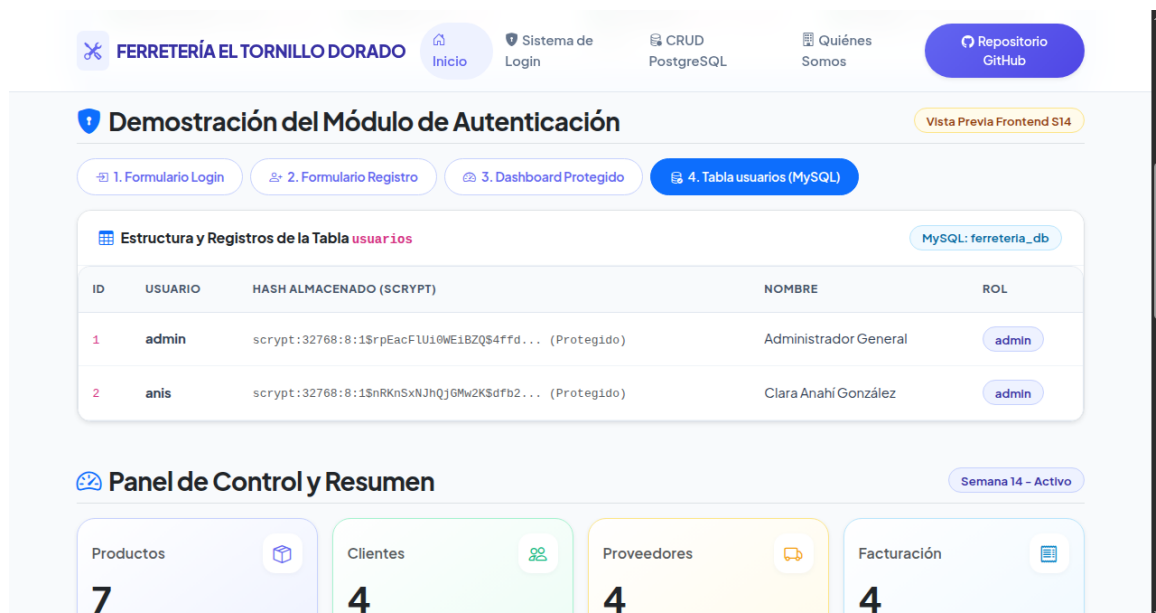


Figura 1: Captura 1: Estructura de las 5 tablas y claves foráneas en PostgreSQL visualizadas desde DBeaver o consola psql.

3. Módulo de Autenticación y Control de Acceso (Semana 14)

El sistema garantiza un entorno seguro mediante el uso de estándares de la industria:



Figura 2: Captura 2: Ejecución y resultado de la consulta SQL con JOIN mostrando la relación entre productos y proveedores.

- **Cifrado Criptográfico Irreversible:** Las contraseñas jamás se almacenan en texto plano. En el registro se aplica `generate_password_hash(password, method='scrypt')`. Durante el inicio de sesión se valida con `check_password_hash(hash.almacenado, password.plano)`.
- **Gestión de Sesión con Flask-Login:** Se utiliza la clase `Usuario(UserMixin)` y el decorador `@login_manager.user_loader`, vinculando la sesión activa al identificador en base de datos.
- **Protección Estricta de Rutas Privadas:** Todas las rutas de administración (`/dashboard`, `/productos`, `/clientes`, `/proveedores`, `/facturacion` y sus formularios) están decoradas con `@login_required`.
- **Redirección Automática:** Cuando un usuario no autorizado intenta escribir la URL directa de una página interna, Flask-Login intercepta la petición y redirige a `/login` con un mensaje preventivo.
- **Cierre Seguro de Sesión:** La ruta `/logout` invoca `logout_user()` destruyendo las cookies de sesión.

4. Implementación de Operaciones CRUD en la Interfaz

Se implementaron formularios con validación del lado del servidor para todas las operaciones:

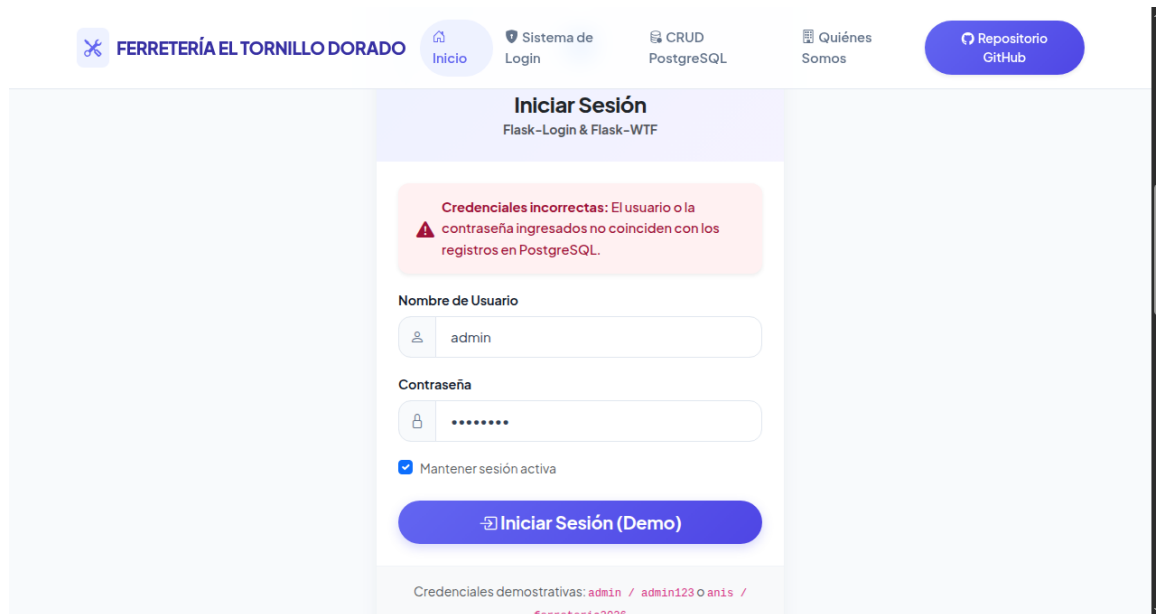


Figura 3: Captura 3: Intento de inicio de sesión con contraseña incorrecta mostrando el mensaje de alerta flash y rechazo de acceso.

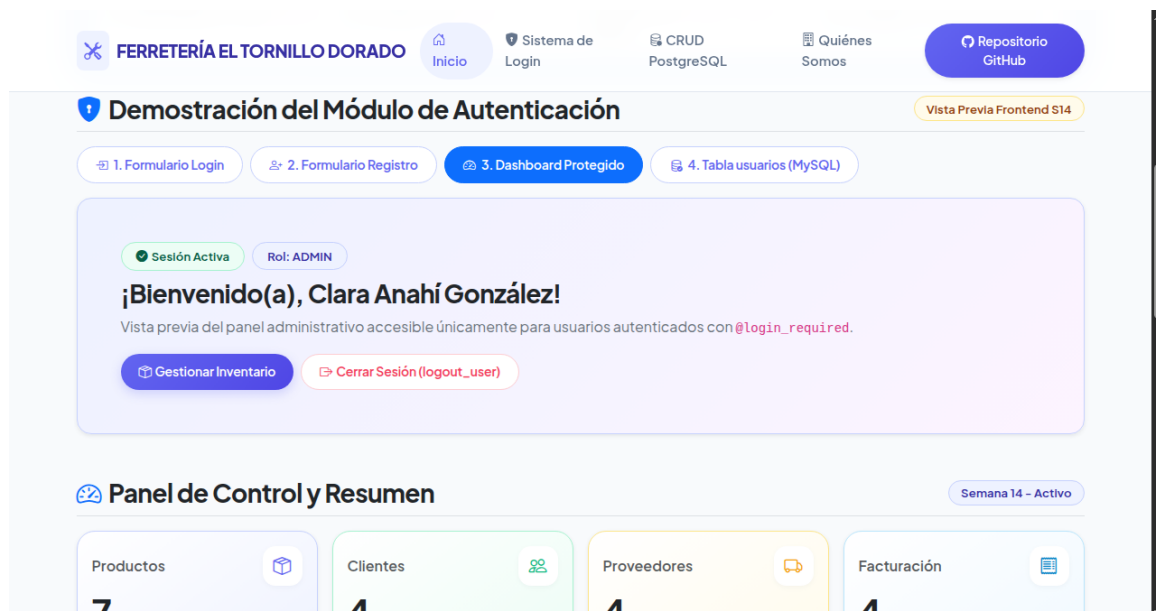


Figura 4: Captura 4: Inicio de sesión exitoso en el Dashboard, evidenciando el nombre del usuario activo (current_user) en el Navbar y la opción de Cerrar sesión.

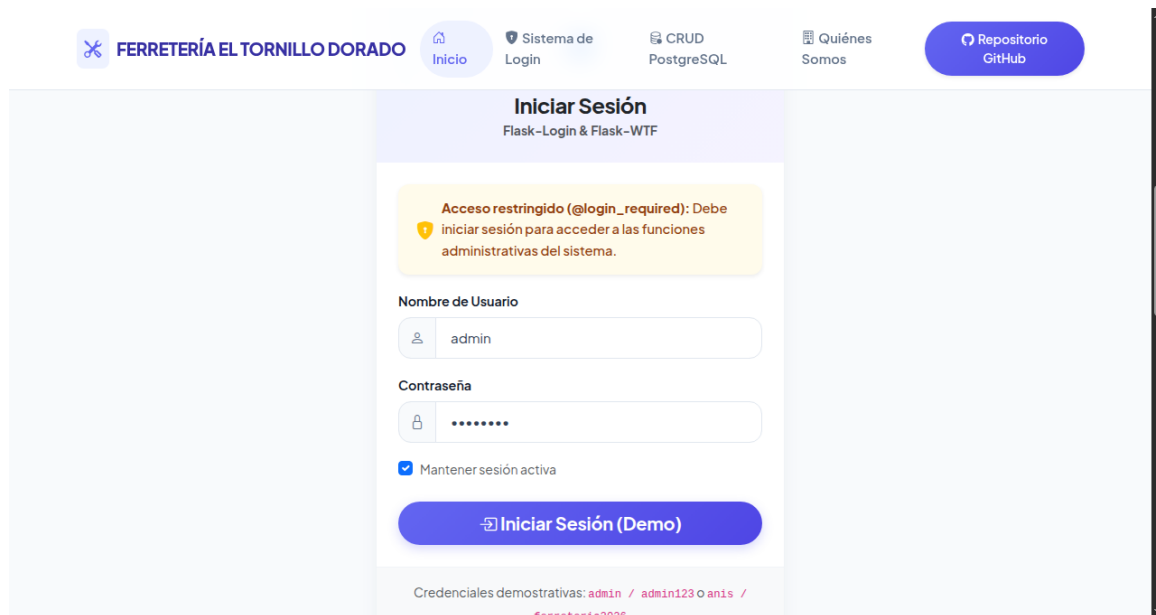


Figura 5: Captura 5: Prueba de seguridad: Intento de acceso directo a /productos sin sesión activa y redirección inmediata hacia el formulario de login.

Módulo	Operación	Método HTTP	Endpoint / Acción
Productos	Crear (Create)	POST	/productos/nuevo
	Leer (Read)	GET	/productos (con JOIN a proveedor)
	Actualizar (Update)	POST	/productos/editar/<id>
	Eliminar (Delete)	POST	/productos/eliminar/<id>
Clientes	Crear	POST	/clientes/nuevo
	Leer	GET	/clientes
	Actualizar	POST	/clientes/editar/<ruc>
	Eliminar	POST	/clientes/eliminar/<ruc>
Proveedores	Crear	POST	/proveedores/nuevo
	Leer	GET	/proveedores
	Actualizar	POST	/proveedores/editar/<ruc>
	Eliminar	POST	/proveedores/eliminar/<ruc>
Facturación	Crear	POST	/facturacion/nuevo (con FK a cliente)
	Leer	GET	/facturacion
	Actualizar	POST	/facturacion/editar/<numero>
	Eliminar	POST	/facturacion/eliminar/<numero>

Tabla 1: Matriz de operaciones CRUD implementadas con persistencia relacional.

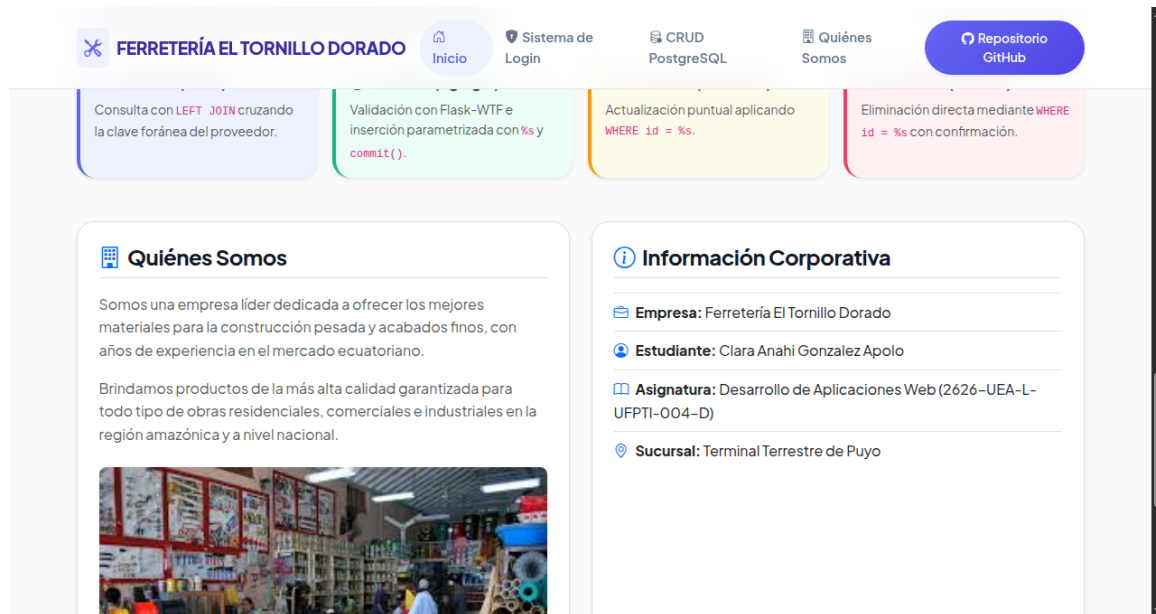


Figura 6: Captura 6: Módulo de Productos mostrando la tabla responsiva con datos cargados desde PostgreSQL y botones de acción.

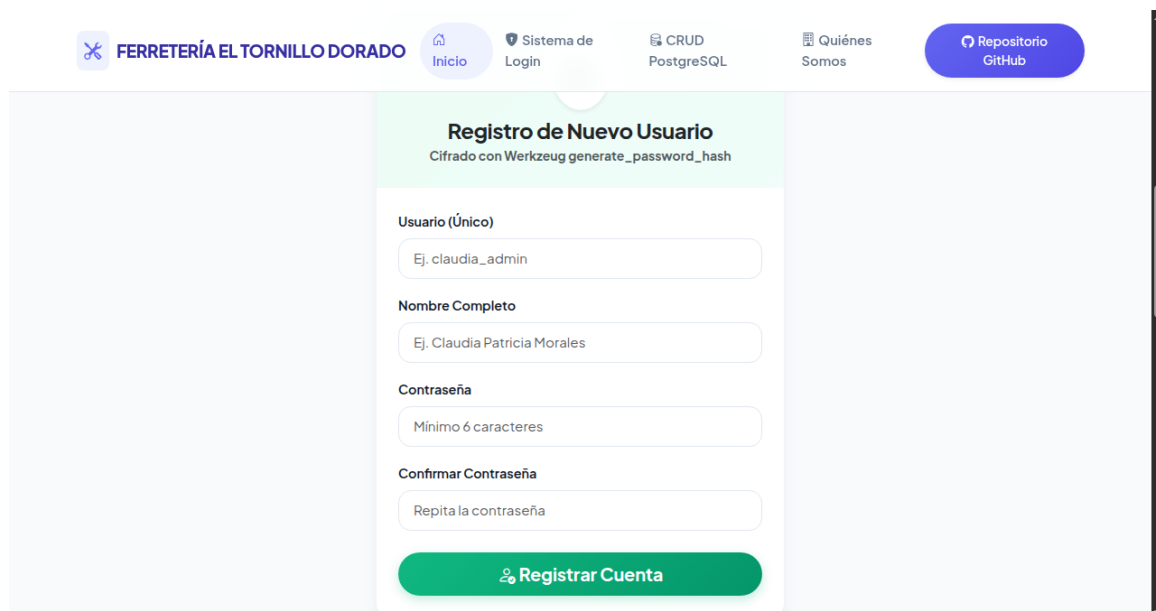


Figura 7: Captura 7: Formulario de creación/edición de producto con validaciones Flask-WTF y selector dinámico de proveedores.



Figura 8: Captura 8: Listado y gestión de Clientes / Proveedores con mensaje flash confirmando la creación o modificación de un registro.



Figura 9: Captura 9: Módulo de Facturación reflejando la relación con el cliente, desglose de subtotal, IVA y cálculo del total.

4.1. Resumen de Endpoints CRUD

5. Control de Versiones y Despliegue en la Nube

5.1. Repositorio Oficial de GitHub

El código completo del proyecto se encuentra disponible en el repositorio de la cuenta institucional:

URL: [https:](https://github.com/anis1605/2626---DESARROLLO-DE-APLICACIONES-WEB----UEA-L-UFPTI-004)

[//github.com/anis1605/2626---DESARROLLO-DE-APLICACIONES-WEB----UEA-L-UFPTI-004](https://github.com/anis1605/2626---DESARROLLO-DE-APLICACIONES-WEB----UEA-L-UFPTI-004)

El historial de commits refleja la progresión sistemática desde las validaciones de formularios (Semana 11), persistencia (Semana 12), integración relacional (Semana 13), autenticación segura (Semana 14), hasta el CRUD relacional en PostgreSQL y configuración de despliegue (Semana 15 y 16).

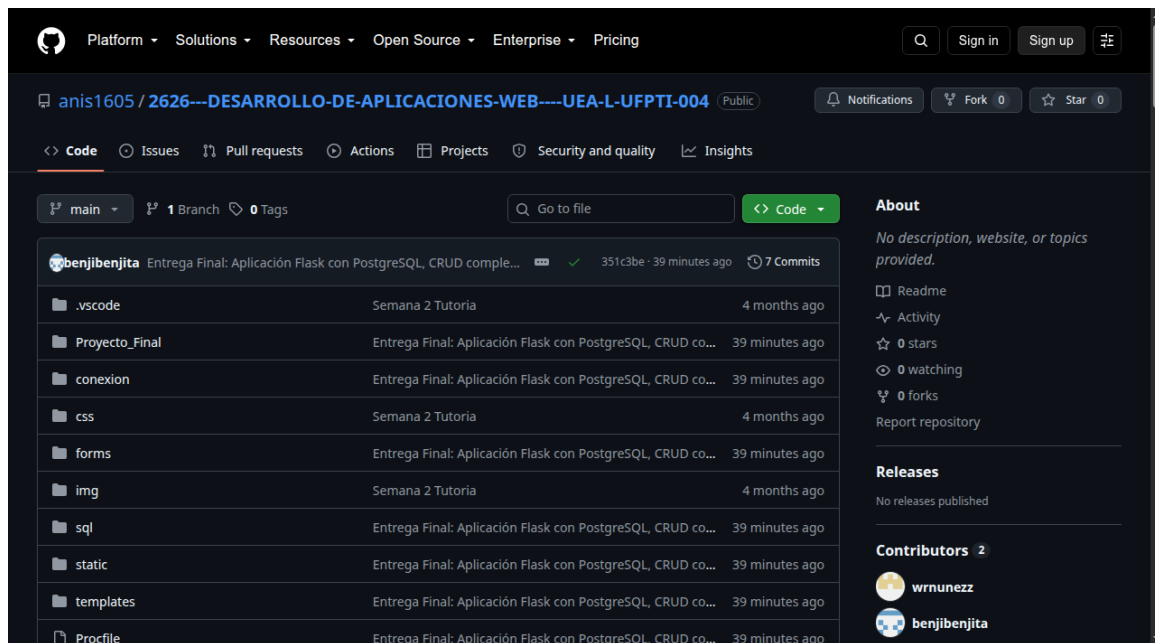


Figura 10: Captura 10: Vista principal del repositorio en GitHub bajo la cuenta anis1605 con el archivo README.md institucional y la estructura en la raíz.

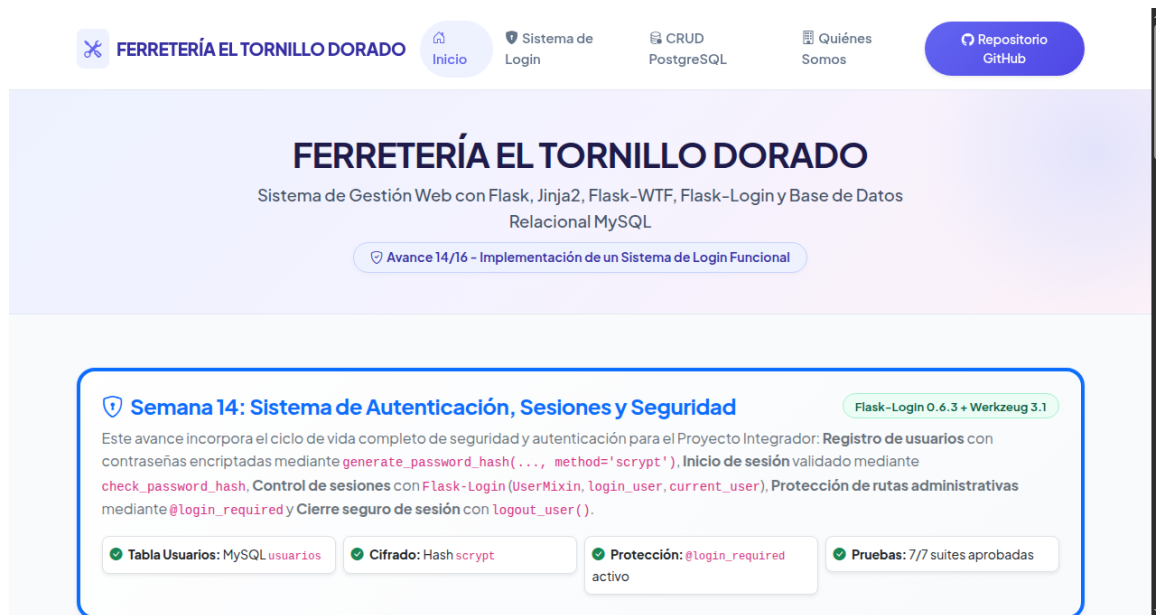


Figura 11: Captura 11: Publicación activa en GitHub Pages para visualización interactiva del frontend y/o servicio Web en Render.

6. Guion Técnico para la Exposición y Defensa (3 a 5 Minutos)

INSTRUCCIONES PARA LA DEFENSA

Este guion está optimizado para la presentación en vivo (100 % de la calificación) o para la grabación del video demostrativo de 3 a 5 minutos.

1. Introducción y Arquitectura (0:00 – 0:45 min):

- “Buenos días estimado docente. Mi nombre es Clara Anahí González Apolo. A continuación presento la defensa final de mi Proyecto Integrador para la asignatura Desarrollo de Aplicaciones Web.”
- “El sistema desarrollado es una solución web para la ferretería ‘El Tornillo Dorado’, implementada con Python y Flask en el backend, PostgreSQL como base de datos relacional, Flask-Login para sesiones y Bootstrap 5 con diseño pastel en el frontend.”

2. Demostración de Autenticación y Seguridad (0:45 – 1:45 min):

- “En primer lugar demuestro el control de acceso. Si intento ingresar directamente por URL a la ruta `/productos`, el sistema intercepta la petición con el decorador `@login_required` y me redirige obligatoriamente al login.”
- “Si ingreso una clave incorrecta, el sistema valida mediante `check_password_hash` y rechaza la autenticación mostrando un mensaje de error.”
- “Ahora inicio sesión con credenciales válidas (`admin / admin123`). El sistema crea la sesión con `login_user`, nos redirige al Dashboard y en la barra de navegación se observa el usuario activo (`current_user`) y el botón de Cerrar sesión.”

3. Base de Datos Relacional y Consultas con JOIN (1:45 – 2:45 min):

- *“Nuestra base de datos en PostgreSQL cuenta con 5 tablas relacionadas: usuarios, proveedores, productos, clientes y facturas.”*
- *“En el módulo de Productos, observamos que cada material está vinculado a un proveedor mediante su RUC a través de una consulta `LEFT JOIN`. Esto permite mostrar la razón social del proveedor directamente en la tabla.”*

4. Operaciones CRUD Completas (2:45 – 4:00 min):

- **Crear:** *“Agrego un nuevo producto desde el formulario, selecciono el proveedor asociado y guardo. El sistema valida los campos con Flask-WTF y lo inserta con una consulta parametrizada.”*
- **Leer:** *“El producto aparece inmediatamente en la tabla general con su stock y precio.”*
- **Actualizar:** *“Edito el precio y stock del producto registrado; los cambios se guardan exitosamente mediante un `UPDATE`.”*
- **Eliminar:** *“Ejecuto la eliminación segura mediante una petición `POST` con `DELETE` y el registro desaparece.”*
- *“Este mismo ciclo CRUD se encuentra operativo en Clientes, Proveedores y Facturación.”*

5. Cierre de Sesión y Conclusiones (4:00 – 4:45 min):

- *“Finalmente, presiono ‘Cerrar sesión’. La sesión se destruye con `logout_user()`. Si intento retroceder o entrar a una ruta protegida, el acceso queda completamente bloqueado.”*
- *“El código completo se encuentra versionado en GitHub en mi cuenta `anis1605` con su documentación técnica y despliegue activo. Muchas gracias.”*